

Introduction au Machine Learning en génomique

Applications à l'antibiorésistance : apprentissage supervisé et non supervisé

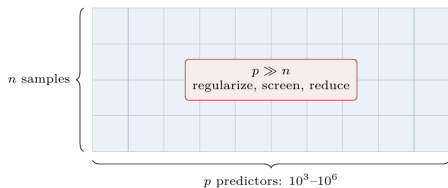
Imad EL BADISY, PhD

11 septembre 2026

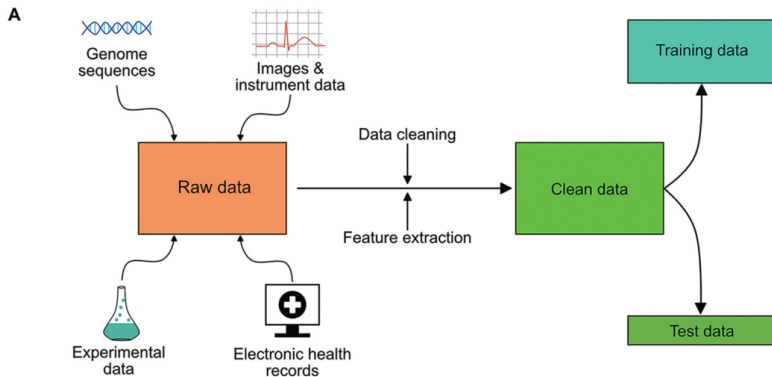
Partie 1 : Machine learning en génomique (applications à l'antibiorésistance)

Pourquoi du Machine learning en génomique ?

- les modèles classiques supposent peu de prédicteurs, choisis à la main
- fortes corrélations (déséquilibre de liaison), interactions, effets non linéaires
- le ML apprend $\hat{f}(x) \rightarrow y$ et **évalue** sur des données non vues (test), fait de la sélection de variables à grande échelle, découvre une structure sans label

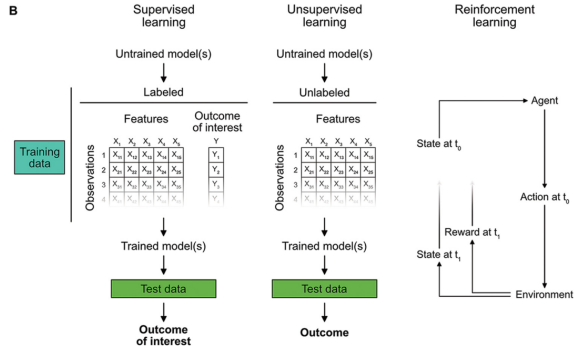


De la donnée brute au modèle entraîné



Anahtar, Yang & Kanjilal (2021), *J. Clin. Microbiol.* 59:e01260-20, Fig. 1A.

Trois types d'apprentissage



Anahtar, Yang & Kanjilal (2021), Fig. 1B. Ce cours couvre les deux types de gauche : **supervisé** (funcml) et **non supervisé** (phynotype) ; l'apprentissage par renforcement est hors périmètre.

Apprentissage supervisé

Données $(x_i, y_i)_{i=1}^n$, on cherche $\hat{f}$ tel que $\hat{f}(x) \approx y$:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i))$$

- y quantitatif $\rightarrow$ régression (RMSE, MAE)
- y qualitatif $\rightarrow$ classification : $\hat{p}(x) = \Pr(y = 1 \mid x)$, puis une classe via un seuil t
- le risque $R(\hat{f}) = \mathbb{E}[L(y, \hat{f}(x))]$ s'estime sur des données **non vues**, jamais sur les données d'entraînement

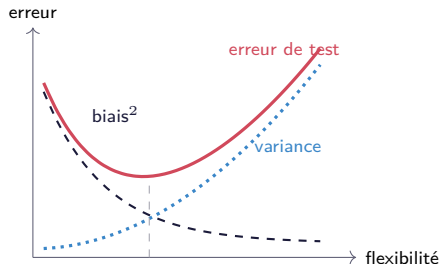
Le compromis biais-variance

Erreur de test = biais² + variance + bruit irréductible

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{biais}^2} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{variance}} + \sigma^2$$

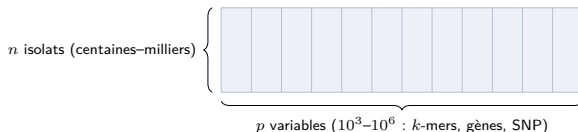
- **trop simple** (biais élevé) :
sous-apprentissage
- **trop flexible** (variance élevée) :
surapprentissage
- objectif : le creux de la courbe d'erreur de **test**

La flexibilité utile est un **hyperparamètre**.



La grande dimension ($p \gg n$) et sa

Le régime typique en génomique, et particulièrement en génomique de l'AMR



- sans contrainte, une infinité de modèles ajustent parfaitement les données d'entraînement ; quand $p \rightarrow \infty$, presque tous les points deviennent équidistants, les notions de voisinage (k -NN, noyaux) se dégradent
- réponses : **régularisation** (lasso/ridge/elastic net), **filtrage**, **réduction de dimension** (PCA, embeddings)

Validation croisée à k blocs

Chaque observation sert une fois à l'évaluation

bloc 1	test	train	train	train	train	→ score ₁	} moyenne ± écart-type
bloc 2	train	test	train	train	train	→ score ₂	
bloc 3	train	train	test	train	train	→ score ₃	
bloc 4	train	train	train	test	train	→ score ₄	
bloc 5	train	train	train	train	test	→ score ₅	

$$CV(\hat{f}) = \frac{1}{v} \sum_{k=1}^v L\left(y^{(k)}, \hat{f}^{(-k)}(x^{(k)})\right)$$

Stratifiée par défaut ; **groupée** quand les observations sont liées.¹

1. Varma & Simon (2006), *BMC Bioinformatics* 7:91.

Cinq pièges qui fragilisent le ML en génomique

- **différences de distribution** : train/test tirés de distributions différentes (lot, espèce, plateforme)
- **exemples dépendants** : isolats liés par une même lignée clonale, viole l'indépendance
- **facteurs de confusion** : la structure de population bactérienne prédit seule la résistance, sans lien causal direct
- **fuite de données** : prétraitement (standardisation, imputation, sélection) ajusté sur l'ensemble complet avant la séparation train/test
- **déséquilibre de classes** : accuracy trompeuse quand une classe est rare

Whalen, Schreiber, Noble & Pollard (2022), *Nat. Rev. Genet.* 23:169–181, “Navigating the pitfalls of applying machine learning in genomics”.

Concepts clés : bactérie et génome

Concept	Définition simple
Escherichia coli (E. coli)	Espèce bactérienne très étudiée. Différents isolats peuvent avoir des profils génétiques différents.
Souche / isolat	Population bactérienne obtenue à partir d'un prélèvement particulier.
Génome bactérien	Ensemble de l'information génétique de la bactérie, principalement portée par l'ADN.
Gène	Séquence d'ADN associée à une fonction biologique.
Mutation / variant	Modification de la séquence d'ADN pouvant influencer un caractère biologique.

Concepts clés : antibiotique et résistance

Concept

Définition simple

Antibiotique

Substance qui tue les bactéries ou freine leur multiplication.

Ciprofloxacin

Antibiotique de la famille des fluoroquinolones étudié dans notre application.

Antibiorésistance (AMR)

Capacité d'une bactérie à résister à l'action d'un antibiotique.

AST

Antimicrobial Susceptibility Testing : test évaluant la sensibilité ou la résistance à un antibiotique.

Génotype / Phénotype

Génotype = information génétique ;
phénotype = caractère observable, ici sensible (S) ou résistant (R).

Apprentissage non supervisé

Les isolats présentent-ils naturellement des profils similaires ?

Caractéristiques des isolats



Clustering



Groupes d'isolats similaires

Objectif

Découvrir la structure naturelle des données sans utiliser le statut sensible/résistant.

Apprentissage supervisé

Peut-on prédire la résistance à la ciprofloxacin ?

Caractéristiques des isolats (X)



Algorithme de classification



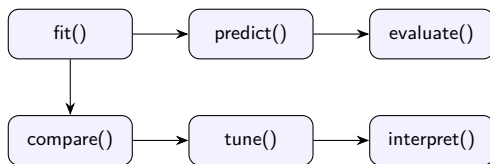
Sensible / Résistant (Y)

Objectif

Prédire le phénotype de résistance d'un nouvel isolat.

Partie 2 : Apprentissage supervisé : comparer plusieurs algorithmes (funcml)

Le pipeline funcml, en un schéma



- `fit` ajuste un apprenant unique ; `compare` ajuste **plusieurs** apprenants sous le même rééchantillonnage
- `tune` règle les hyperparamètres par validation croisée ; `interpret` explique le modèle ajusté (importance, effets)

Les familles d'algorithmes

- **linéaires / pénalisés** : `glm` (référence interprétable), `glmnet` (lasso/ridge pour $p \gg n$)
- **arbres uniques** : `rpart` (CART, glouton), `ctree` (test d'indépendance formel)
- **forêts** : `ranger/randomForest` (bagging + sous-espace aléatoire), `cforest`
- **boosting** : `gbm`, `xgboost` (second ordre + régularisation), `lightgbm`, `adaboost`
- **distance / marge** : `kkn` (k plus proches voisins), `e1071_svm` (marge maximale, noyau)
- **additifs / splines** : `gam`, `earth` (MARS)
- **réseaux de neurones** : `dmlp`, `mlp`, `nnet`
- **ensembles** : `stacking`, `superlearner`, combinaison pondérée, réglée par validation croisée

Nos données : résistance à la ciprofloxacin

- `data/ecoli_amr_isolate_v2.csv` : 700 isolats d'*E. coli*, 8 antibiotiques testés
- cible retenue : **ciprofloxacin**, seule colonne sans donnée manquante (700/700), et bien répartie (144 résistants / 539 sensibles)
- prédictors génomiques disponibles pour tous les isolats : `genome_length`, `gc_content`, `contigs`, `collection_year`
- `collection_year` comporte des valeurs manquantes (16 %) → imputation nécessaire avant modélisation

Comparer plusieurs algorithmes avec `compare()`

Même rééchantillonnage, plusieurs modèles à la fois, la bonne façon de choisir un algorithme, plutôt que d'en essayer un seul :

```
compare_obj <- compare(  
  data = model_dat,  
  formula = ciprofloxacin ~ genome_length + gc_content +  
    contigs + collection_year,  
  models = c("glm", "rpart", "ranger", "xgboost", "kknn", "e1071_svm"),  
  metrics = c("accuracy", "auc", "f1", "balanced_accuracy"),  
  resampling = cv(v = 5, seed = 42), seed = 42  
)
```

Comment choisir entre plusieurs modèles

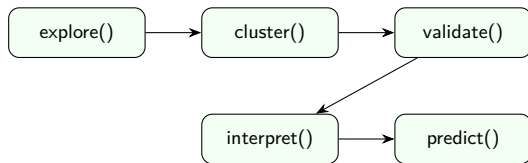
- `compare()` renvoie, pour chaque modèle $\times$ métrique, une moyenne **et** un intervalle de confiance sur les plis de validation croisée
- on retient le modèle sur l'**intervalle**, pas seulement sur la moyenne : deux modèles dont les intervalles se chevauchent ne sont pas franchement différents
- un arbre unique (`rpart`) est en général instable d'un pli à l'autre ; forêts et boosting réduisent cette variance en moyennant plusieurs arbres

Au-delà de la discrimination

- l'AUC mesure la capacité à **classer** les cas devant les témoins, elle ne dit rien de la calibration (les probabilités prédites sont-elles fiables ?)
- `roc_curve()` visualise le compromis sensibilité/spécificité sur toute la gamme de seuils, pas seulement au seuil $t = 0,5$
- `interpret(fit, method = "permute")` : on permute chaque variable et on mesure la perte de performance, plus la chute est grande, plus la variable compte pour le modèle ajusté (Watson, 2022, *Hum. Genet.* 141:1499–1513)

Partie 3 : Apprentissage non supervisé : comparer plusieurs algorithmes de clustering (phynotype)

Le pipeline phynotype, en un schéma



- `explore` projette les données (ACP) pour repérer une structure visuelle avant de fixer une méthode
- `cluster` ajuste une méthode pour un k donné ; `validate` compare plusieurs méthodes $\times$ plusieurs k sur les mêmes indices

Panorama des méthodes de clustering

- **k-means** : minimise la somme des carrés intra-cluster $\sum_j \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$, rapide, suppose des clusters sphériques
- **PAM** (Partitioning Around Medoids) : comme *k*-means mais centré sur des observations réelles, robuste aux valeurs aberrantes
- **hclust** / **agnes** : clustering hiérarchique, agglomératif, pas besoin de fixer *k* à l'avance, mais sensible aux petits groupes atypiques
- **DBSCAN** : basé sur la densité, trouve le nombre de clusters automatiquement, isole le bruit
- **GMM** : mélange de gaussiennes, affectation probabiliste (soft clustering)
- **kproto** / **KMM** : pour données mixtes (numériques + catégorielles)

Indices de validation interne

$$\text{Silhouette : } s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1] \quad \text{Calinski-Harabasz : } CH = \frac{\text{BSS}/(k-1)}{\text{WSS}/(n-k)}$$

$$\text{Davies-Bouldin : } DB = \frac{1}{k} \sum_{j=1}^k \max_{l \neq j} \frac{s_j + s_l}{d(\mu_j, \mu_l)}$$

- silhouette et CH : **plus haut** = meilleur ; Davies-Bouldin : **plus bas** = meilleur
- ces trois indices mesurent la séparation/compacité, **sans** connaître de vraie étiquette

Comparer plusieurs méthodes $\times$ plusieurs valeurs de k

`validate()` ajuste chaque méthode enregistrée pour une grille de k , sur les **mêmes** variables standardisées, et renvoie les trois indices pour chaque configuration :

```
validate(  
  data = model_dat,  
  vars = c("genome_length", "gc_content", "contigs", "collection_year"),  
  methods = c("kmeans", "pam", "hclust", "gmm"),  
  k = 2:5  
)
```

- une seule table à lire pour comparer méthodes et k , plutôt qu'un ajustement à la fois

Piège : un indice élevé peut cacher un clustering dégénéré

- un clustering hiérarchique peut afficher la **meilleure silhouette de toute la grille** en isolant seulement 2-3 observations atypiques dans un “cluster” à part
- un indice interne élevé peut donc signaler des points aberrants isolés, pas une vraie structure dans les données
- règle générale : **exclude** toute configuration où un cluster contient moins d'un seuil raisonnable des observations (p. ex. 5 %) avant de choisir sur la silhouette
- même logique que la comparaison d'algorithmes supervisés : ne jamais choisir sur un seul chiffre sans vérifier ce qu'il mesure réellement

Choisir la configuration finale

- parmi les configurations à tailles de cluster équilibrées, on retient celle avec le meilleur compromis silhouette / Calinski-Harabasz / Davies-Bouldin
- `plot_clusters()` : projette les clusters retenus en 2D (ACP) pour une inspection visuelle
- `plot_silhouette()` : une barre par observation, une silhouette négative signale une observation mal classée par rapport à son cluster
- `interpret()` : caractérise chaque cluster par ses variables les plus discriminantes